中山大学计算机院本科生实验报告

(2025 学年春季学期)

课程名称：并行程序设计　　批改人：

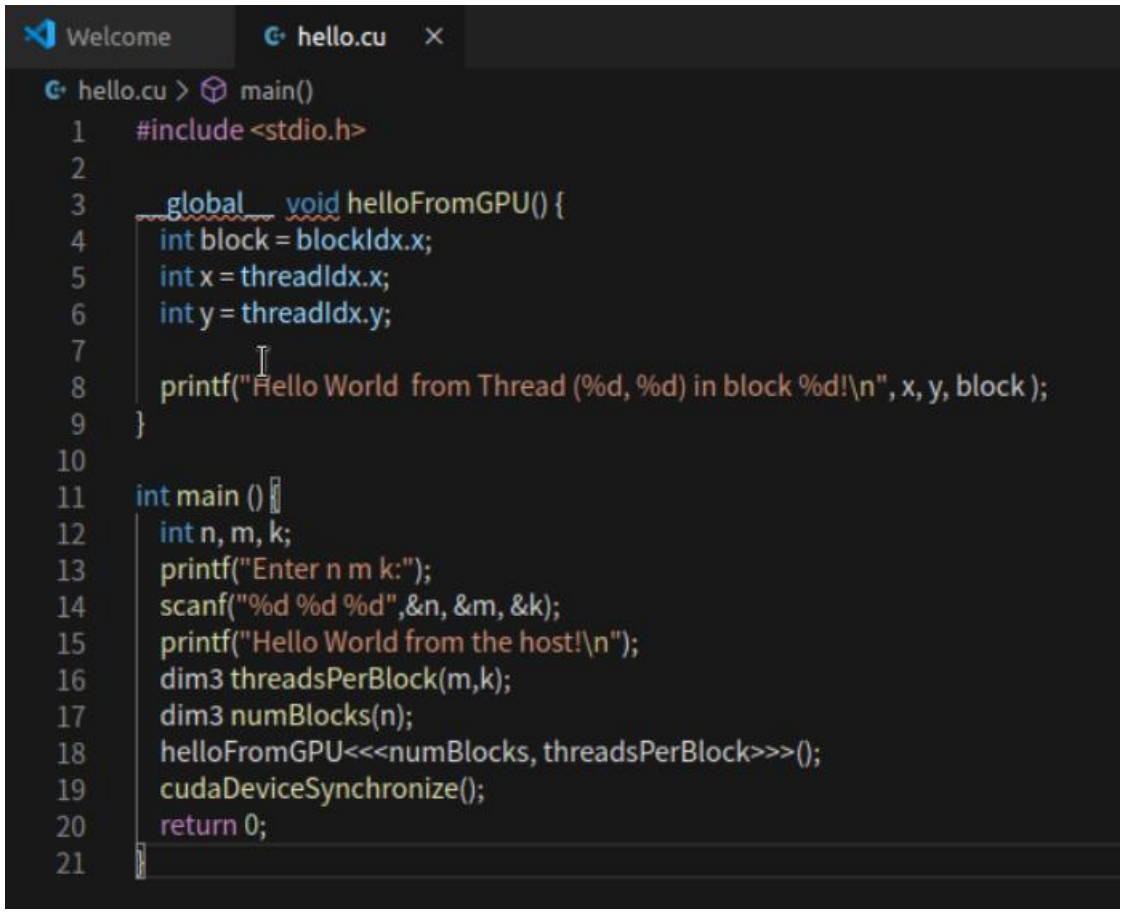
实验	CUDA 矩阵转置	专业（方向）	计算机科学与技术
学号	22320021	姓名	陈安康
Email	1743826979@qq.com	完成日期	2025/5/31

1. 实验目的

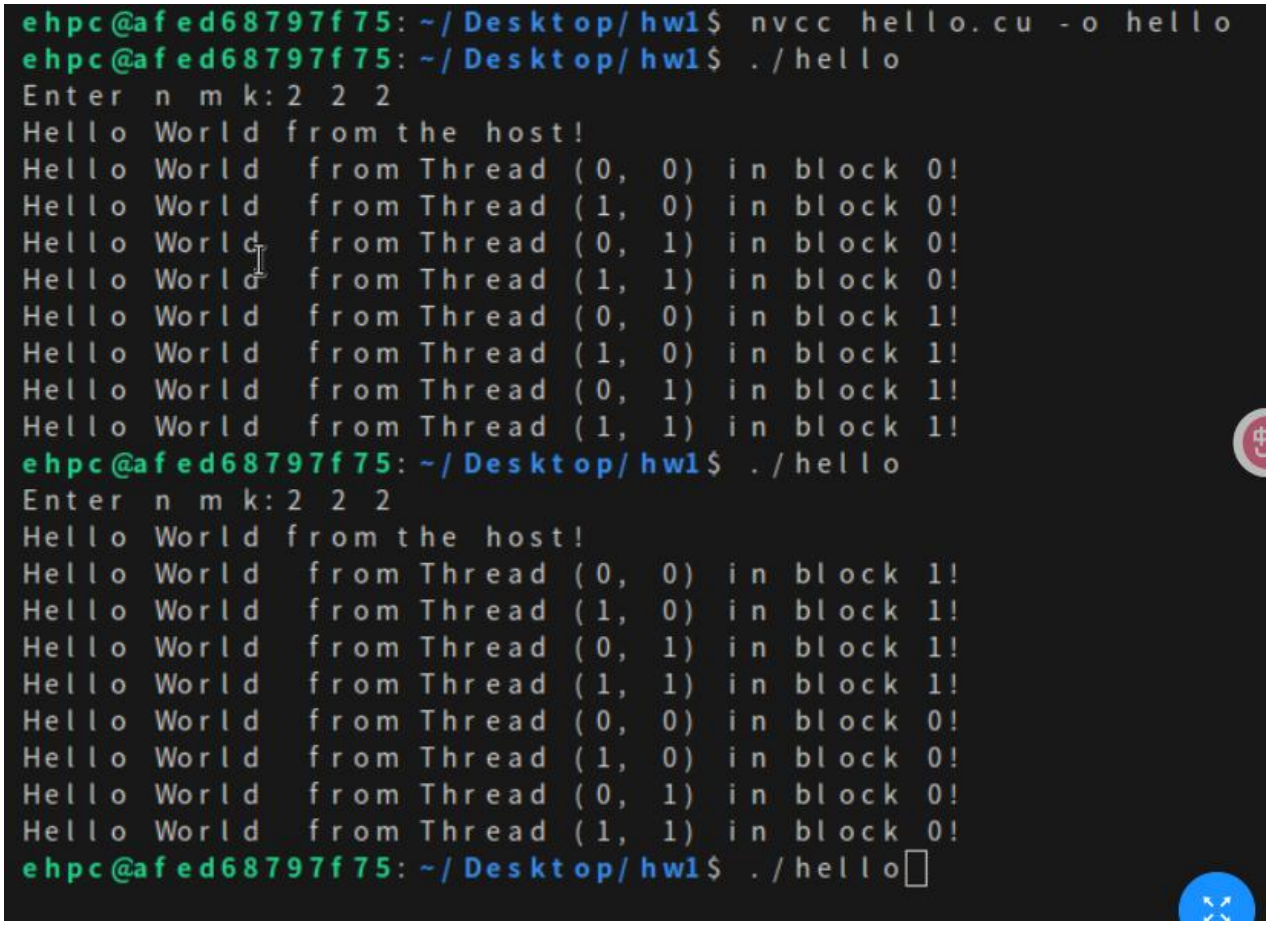
练习 CUDA 编程，学习 CUDA 编程的原理以及 GPU 的架构对编程性能的影响。

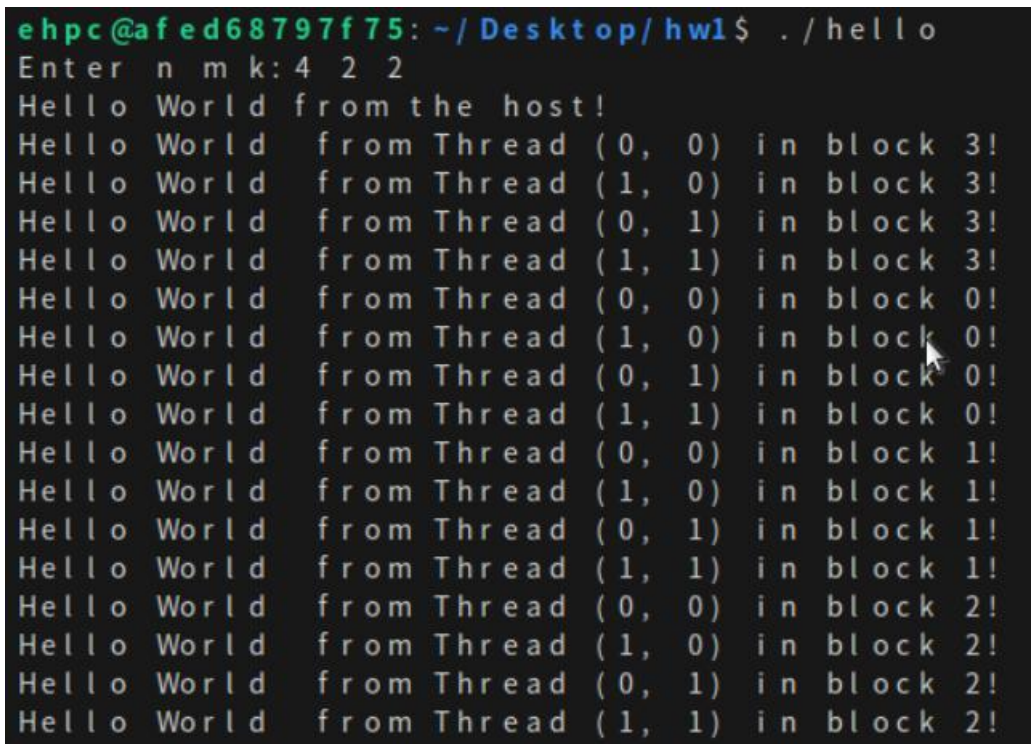
2. 实验过程和核心代码

CUDA 编程需要 CPU 和 GPU 协作，host 指代 CPU 及其内存，而用 device 指代 GPU 及其内存。CUDA 程序中既包含 host 程序，又包含 device 程序，它们分别在 CPU 和 GPU 上运行。同时，host 与 device 之间可以进行通信。核函数是在 device 上线程中并行执行的函数，核函数通过__global__符号声明。调用时需要用<<<grid, block>>>来指定 kernel 要执行的线程数量。下面是 hello 程序，用来在 host 端和 device 端打印 hello 字符串：



运行结果如下：

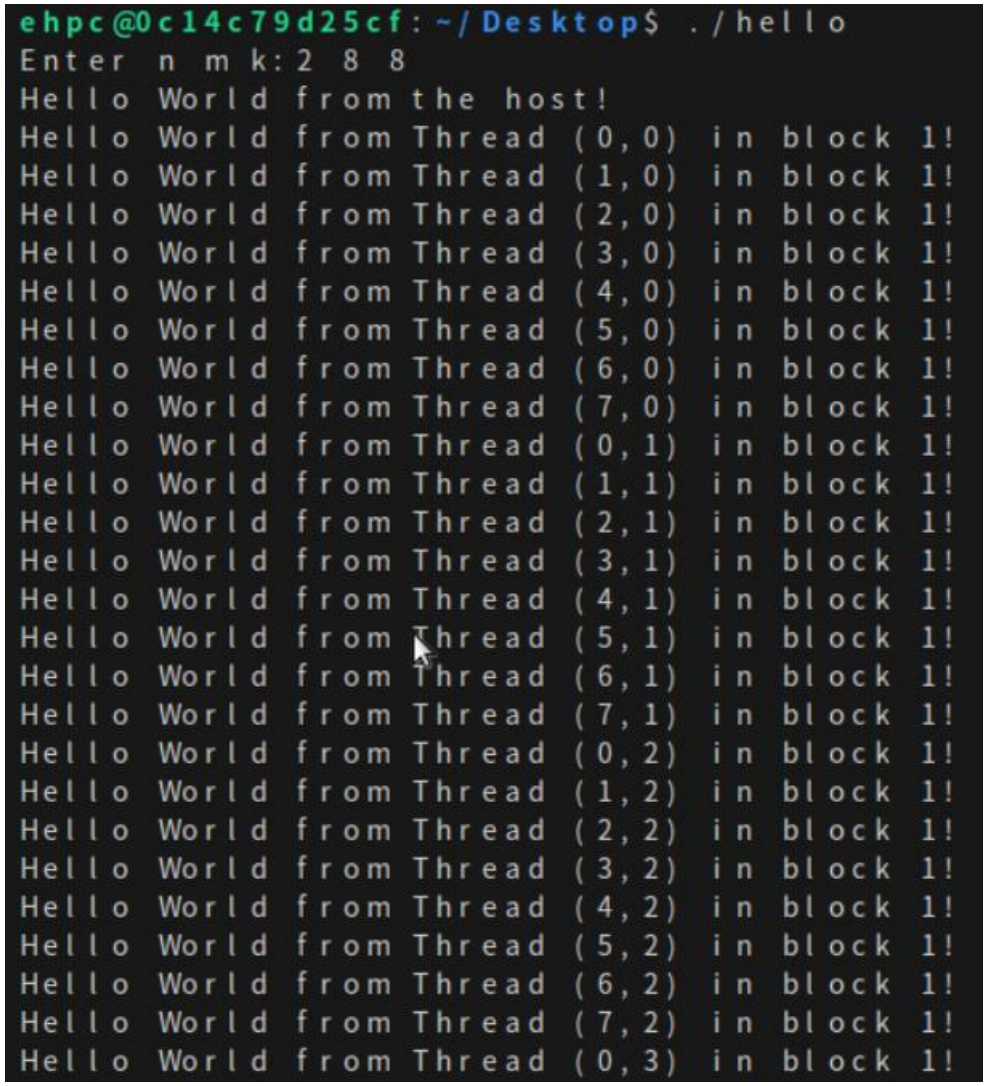

ehpc@afed68797f75:~/Desktop/hw1$ nvcc hello.cu -o hello
ehpc@afed68797f75:~/Desktop/hw1$ ./hello
Enter n m k:2 2 2
Hello World from the host!
Hello World from Thread (0, 0) in block 0!
Hello World from Thread (1, 0) in block 0!
Hello World from Thread (0, 1) in block 0!
Hello World from Thread (1, 1) in block 0!
Hello World from Thread (0, 0) in block 1!
Hello World from Thread (1, 0) in block 1!
Hello World from Thread (0, 1) in block 1!
Hello World from Thread (1, 1) in block 1!
ehpc@afed68797f75:~/Desktop/hw1$ ./hello
Enter n m k:2 2 2
Hello World from the host!
Hello World from Thread (0, 0) in block 1!
Hello World from Thread (1, 0) in block 1!
Hello World from Thread (0, 1) in block 1!
Hello World from Thread (1, 1) in block 1!
Hello World from Thread (0, 0) in block 0!
Hello World from Thread (1, 0) in block 0!
Hello World from Thread (0, 1) in block 0!
Hello World from Thread (1, 1) in block 0!
ehpc@afed68797f75:~/Desktop/hw1$ ./hello


ehpc@afed68797f75:~/Desktop/hw1$ ./hello
Enter n m k:4 2 2
Hello World from the host!
Hello World from Thread (0, 0) in block 3!
Hello World from Thread (1, 0) in block 3!
Hello World from Thread (0, 1) in block 3!
Hello World from Thread (1, 1) in block 3!
Hello World from Thread (0, 0) in block 0!
Hello World from Thread (1, 0) in block 0!
Hello World from Thread (0, 1) in block 0!
Hello World from Thread (1, 1) in block 0!
Hello World from Thread (0, 0) in block 1!
Hello World from Thread (1, 0) in block 1!
Hello World from Thread (0, 1) in block 1!
Hello World from Thread (1, 1) in block 1!
Hello World from Thread (0, 0) in block 2!
Hello World from Thread (1, 0) in block 2!
Hello World from Thread (0, 1) in block 2!
Hello World from Thread (1, 1) in block 2!

```
ehpc@afed68797f75:~/Desktop/hw1$ ./hello
Enter n m k: 4 2 2
Hello World from the host!
Hello World  from Thread (0, 0) in block 3!
Hello World  from Thread (1, 0) in block 3!
Hello World  from Thread (0, 1) in block 3!
Hello World  from Thread (1, 1) in block 3!
Hello World  from Thread (0, 0) in block 0!
Hello World  from Thread (1, 0) in block 0!
Hello World  from Thread (0, 1) in block 0!
Hello World  from Thread (1, 1) in block 0!
Hello World  from Thread (0, 0) in block 2!
Hello World  from Thread (1, 0) in block 2!
Hello World  from Thread (0, 1) in block 2!
Hello World  from Thread (1, 1) in block 2!
Hello World  from Thread (0, 0) in block 1!
Hello World  from Thread (1, 0) in block 1!
Hello World  from Thread (0, 1) in block 1!
Hello World  from Thread (1, 1) in block 1!
```

```
ehpc@afed68797f75:~/Desktop/hw1$ ./hello
Enter n m k: 4 2 2
Hello World from the host!
Hello World  from Thread (0, 0) in block 3!
Hello World  from Thread (1, 0) in block 3!
Hello World  from Thread (0, 1) in block 3!
Hello World  from Thread (1, 1) in block 3!
Hello World  from Thread (0, 0) in block 2!
Hello World  from Thread (1, 0) in block 2!
Hello World  from Thread (0, 1) in block 2!
Hello World  from Thread (1, 1) in block 2!
Hello World  from Thread (0, 0) in block 1!
Hello World  from Thread (1, 0) in block 1!
Hello World  from Thread (0, 1) in block 1!
Hello World  from Thread (1, 1) in block 1!
Hello World  from Thread (0, 0) in block 0!
Hello World  from Thread (1, 0) in block 0!
Hello World  from Thread (0, 1) in block 0!
Hello World  from Thread (1, 1) in block 0!
```

将块内的线程数量增大至大于 32，再次执行，会发现块内输出也不有序了，但每 32 个线程之间还是有序的：


ehpc@0c14c79d25cf:~/Desktop$ ./hello
Enter n m k: 2 8 8
Hello World from the host!
Hello World from Thread (0,0) in block 1!
Hello World from Thread (1,0) in block 1!
Hello World from Thread (2,0) in block 1!
Hello World from Thread (3,0) in block 1!
Hello World from Thread (4,0) in block 1!
Hello World from Thread (5,0) in block 1!
Hello World from Thread (6,0) in block 1!
Hello World from Thread (7,0) in block 1!
Hello World from Thread (0,1) in block 1!
Hello World from Thread (1,1) in block 1!
Hello World from Thread (2,1) in block 1!
Hello World from Thread (3,1) in block 1!
Hello World from Thread (4,1) in block 1!
Hello World from Thread (5,1) in block 1!
Hello World from Thread (6,1) in block 1!
Hello World from Thread (7,1) in block 1!
Hello World from Thread (0,2) in block 1!
Hello World from Thread (1,2) in block 1!
Hello World from Thread (2,2) in block 1!
Hello World from Thread (3,2) in block 1!
Hello World from Thread (4,2) in block 1!
Hello World from Thread (5,2) in block 1!
Hello World from Thread (6,2) in block 1!
Hello World from Thread (7,2) in block 1!
Hello World from Thread (0,3) in block 1!

```
Hello World from Thread (0,3) in block 1!
Hello World from Thread (1,3) in block 1!
Hello World from Thread (2,3) in block 1!
Hello World from Thread (3,3) in block 1!
Hello World from Thread (4,3) in block 1!
Hello World from Thread (5,3) in block 1!
Hello World from Thread (6,3) in block 1!
Hello World from Thread (7,3) in block 1!
Hello World from Thread (0,0) in block 0!
Hello World from Thread (1,0) in block 0!
Hello World from Thread (2,0) in block 0!
Hello World from Thread (3,0) in block 0!
Hello World from Thread (4,0) in block 0!
Hello World from Thread (5,0) in block 0!
Hello World from Thread (6,0) in block 0!
Hello World from Thread (7,0) in block 0!
Hello World from Thread (0,1) in block 0!
Hello World from Thread (1,1) in block 0!
Hello World from Thread (2,1) in block 0!
Hello World from Thread (3,1) in block 0!
Hello World from Thread (4,1) in block 0!
Hello World from Thread (5,1) in block 0!
Hello World from Thread (6,1) in block 0!
Hello World from Thread (7,1) in block 0!
Hello World from Thread (0,2) in block 0!
Hello World from Thread (1,2) in block 0!
Hello World from Thread (2,2) in block 0!
Hello World from Thread (3,2) in block 0!
```

```
Hello World from Thread (4,2) in block 0!
Hello World from Thread (5,2) in block 0!
Hello World from Thread (6,2) in block 0!
Hello World from Thread (7,2) in block 0!
Hello World from Thread (0,3) in block 0!
Hello World from Thread (1,3) in block 0!
Hello World from Thread (2,3) in block 0!
Hello World from Thread (3,3) in block 0!
Hello World from Thread (4,3) in block 0!
Hello World from Thread (5,3) in block 0!
Hello World from Thread (6,3) in block 0!
Hello World from Thread (7,3) in block 0!
Hello World from Thread (0,4) in block 1!
Hello World from Thread (1,4) in block 1!
Hello World from Thread (2,4) in block 1!
Hello World from Thread (3,4) in block 1!
Hello World from Thread (4,4) in block 1!
Hello World from Thread (5,4) in block 1!
Hello World from Thread (6,4) in block 1!
Hello World from Thread (7,4) in block 1!
Hello World from Thread (0,5) in block 1!
Hello World from Thread (1,5) in block 1!
Hello World from Thread (2,5) in block 1!
Hello World from Thread (3,5) in block 1!
Hello World from Thread (4,5) in block 1!
Hello World from Thread (5,5) in block 1!
Hello World from Thread (6,5) in block 1!
Hello World from Thread (7,5) in block 1!
```

```
Hello World from Thread (0,6) in block 1!
Hello World from Thread (1,6) in block 1!
Hello World from Thread (2,6) in block 1!
Hello World from Thread (3,6) in block 1!
Hello World from Thread (4,6) in block 1!
Hello World from Thread (5,6) in block 1!
Hello World from Thread (6,6) in block 1!
Hello World from Thread (7,6) in block 1!
Hello World from Thread (0,7) in block 1!
Hello World from Thread (1,7) in block 1!
Hello World from Thread (2,7) in block 1!
Hello World from Thread (3,7) in block 1!
Hello World from Thread (4,7) in block 1!
Hello World from Thread (5,7) in block 1!
Hello World from Thread (6,7) in block 1!
Hello World from Thread (7,7) in block 1!
Hello World from Thread (0,4) in block 0!
Hello World from Thread (1,4) in block 0!
Hello World from Thread (2,4) in block 0!
Hello World from Thread (3,4) in block 0!
Hello World from Thread (4,4) in block 0!
Hello World from Thread (5,4) in block 0!
Hello World from Thread (6,4) in block 0!
Hello World from Thread (7,4) in block 0!
Hello World from Thread (0,5) in block 0!
Hello World from Thread (1,5) in block 0!
Hello World from Thread (2,5) in block 0!
Hello World from Thread (3,5) in block 0!
```

```
Hello World from Thread (4,5) in block 0!
Hello World from Thread (5,5) in block 0!
Hello World from Thread (6,5) in block 0!
Hello World from Thread (7,5) in block 0!
Hello World from Thread (0,6) in block 0!
Hello World from Thread (1,6) in block 0!
Hello World from Thread (2,6) in block 0!
Hello World from Thread (3,6) in block 0!
Hello World from Thread (4,6) in block 0!
Hello World from Thread (5,6) in block 0!
Hello World from Thread (6,6) in block 0!
Hello World from Thread (7,6) in block 0!
Hello World from Thread (0,7) in block 0!
Hello World from Thread (1,7) in block 0!
Hello World from Thread (2,7) in block 0!
Hello World from Thread (3,7) in block 0!
Hello World from Thread (4,7) in block 0!
Hello World from Thread (5,7) in block 0!
Hello World from Thread (6,7) in block 0!
Hello World from Thread (7,7) in block 0!
```

更大规模的打印信息过多，不方便展示，从多次实验中可以看到，每 32 个线程（一个 warp）输出是有序的，不同 warp 之间的输出顺序是随机的。

grid 和 block 都是逻辑划分，在真正执行时，cuda 以 warp 为单位进行执行，warp 是 CUDA 的物理执行单位，一个 warp 包含 32 个 CUDA 线程，这 32 个线程是作为一个单元一起发射指令。warp 的执行模型是 SIMT（单指令多线程），32 个线程会同步执行相同指令。一个线程块由一个或者多个 wrap 组成。线程块内的线程执行在同一 SM 中，且会被进一步分成 warp 执行。由于 printf 输出缓冲区按照线程索引顺序管理，所以同一个 warp 中的线程的输出是有序的，而各 warp 是并行的，所以 warp 之间的输出顺序是无序的。

矩阵转置的代码实现如下，这里我让每个线程负责一个元素的转置，在 Global Memory 中存储矩阵，实现代码如下，优化放在实验结果部分：

```cpp
#include <stdio.h>
#include <cuda_runtime.h>
#include <stdlib.h>
#include <time.h>

#define IDX(i, j, n) ((i) * (n) + (j))

__global__ void transposeKernel(float* A, float* AT, int n) {
    int i = blockIdx.y * blockDim.y + threadIdx.y;  // 行
    int j = blockIdx.x * blockDim.x + threadIdx.x;  // 列
    if (i < n && j < n) {
        AT[IDX(j, i, n)] = A[IDX(i, j, n)];
    }
}

void fillMatrix(float* A, int n) {
    for (int i = 0; i < n * n; i++) {
        A[i] = static_cast<float>(rand()) / RAND_MAX;
    }
}

void printMatrix(float* A, int n, const char* name) {
    printf("%s:\n", name);
    for (int i = 0; i < n && i < 8; i++) {
        for (int j = 0; j < n && j < 8; j++) {
            printf("%6.2f ", A[IDX(i, j, n)]);
        }
        printf("\n");
    }
    printf("...\n");
}
```

```
int main(int argc, char** argv) {
    if (argc != 2) {
        printf("Usage: %s <n>\n", argv[0]);
        return 1;
    }
    int n = atoi(argv[1]);
    size_t size = n * n * sizeof(float);

    float* h_A = (float*)malloc(size);
    float* h_AT = (float*)malloc(size);
    fillMatrix(h_A, n);

    float *d_A, *d_AT;
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_AT, size);
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);

    int blockDimx = 16;
    int blockDimy = 16;
    dim3 blockDim(blockDimx, blockDimy);
    dim3 gridDim((n + blockDimx - 1) / blockDimx, (n + blockDimy - 1) / blockDimy);

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaEventRecord(start);
    transposeKernel<<<gridDim, blockDim>>>(d_A, d_AT, n);
    cudaEventRecord(stop);

    cudaMemcpy(h_AT, d_AT, size, cudaMemcpyDeviceToHost);
    cudaDeviceSynchronize();

    float milliseconds;
    cudaEventElapsedTime(&milliseconds, start, stop);

    printMatrix(h_A, n, "Matrix A");
```

3. 实验结果

由于权限限制，我无法使用分析工具来分析详细的性能。

在 block 为（4,8）的情况下运行：

```
ehpc@c4f0c97b3278:~/Desktop$ ./mt 512
Transpose time: 0.1715 ms
ehpc@c4f0c97b3278:~/Desktop$ ./mt 1024
Transpose time: 0.3684 ms
ehpc@c4f0c97b3278:~/Desktop$ ./mt 2048
Transpose time: 1.1628 ms
```

在 block 为（8,8）的情况下运行：

```
ehpc@c4f0c97b3278:~/Desktop$ ./mt 512
Transpose time: 0.1456 ms
ehpc@c4f0c97b3278:~/Desktop$ ./mt 1024
Transpose time: 0.2724 ms
ehpc@c4f0c97b3278:~/Desktop$ ./mt 2048
Transpose time: 0.7257 ms
```

在 block 为（16,8）的情况下运行：

```
ehpc@c4f0c97b3278:~/Desktop$ ./mt 512
Transpose time: 0.1513 ms
ehpc@c4f0c97b3278:~/Desktop$ ./mt 1024
Transpose time: 0.2649 ms
ehpc@c4f0c97b3278:~/Desktop$ ./mt 2048
Transpose time: 0.6512 ms
```

在 block 为（16,16）的情况下运行：

```
ehpc@c4f0c97b3278:~/Desktop$ ./mt 512
Transpose time: 0.1506 ms
ehpc@c4f0c97b3278:~/Desktop$ ./mt 1024
Transpose time: 0.2428 ms
ehpc@c4f0c97b3278:~/Desktop$ ./mt 2048
Transpose time: 0.6159 ms
```

在 block 为（32,32）的情况下运行：

```
ehpc@c4f0c97b3278:~/Desktop$ ./mt 512
Transpose time: 0.1923 ms
ehpc@c4f0c97b3278:~/Desktop$ ./mt 1024
Transpose time: 0.3223 ms
ehpc@c4f0c97b3278:~/Desktop$ ./mt 2048
Transpose time: 0.8780 ms
```

通过上面的结果可以看到，矩阵规模越大，运行时间越长。同时 block 大小对于性能影响十分大，由于 warp 的大小为 32，所以在设计 block 大小时尽量设置为 32 的倍数。以下是分别使用（64,8）和（65,8）block 运行的结果，如下：

（64,8）block：

```
Transpose time:  0.8935  ms
```

（65,8）block：

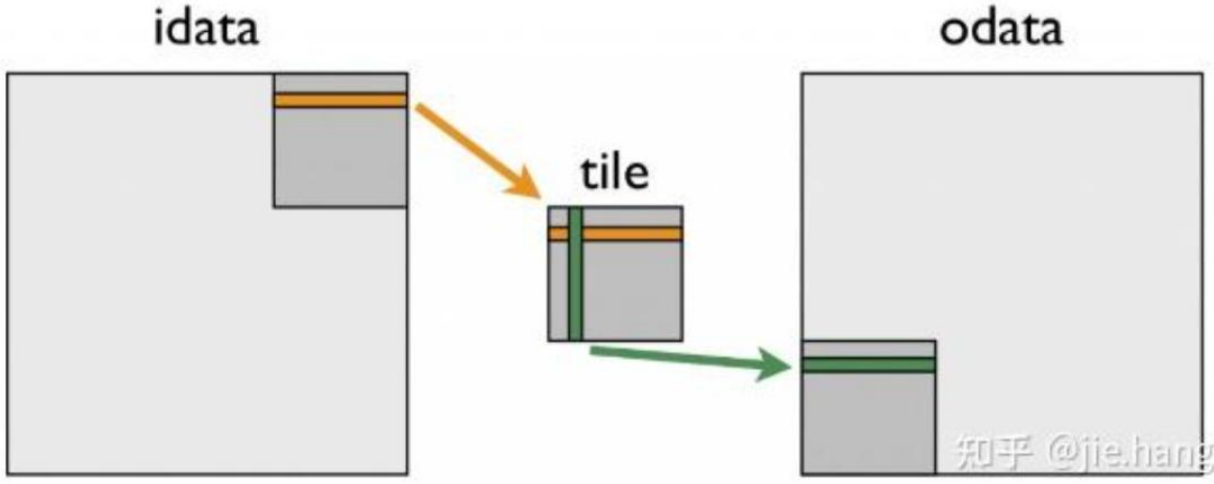
```
Transpose time:  0.9950  ms
```

由此可见，很小的参数修改就会造成很大的性能损失。由于（65,8）的 block 不是 32 的倍数，导致划分到 warp 时会有填不满的 warp，造成额外性能损失。

从上面的实验可以看到，block 并不是越大越好，也不是越小越好，当前性能表现最好的 block 出现在（16,8）时，因此在 CUDA 编程中，依据问题和 GPU 来合理选择 block 大小至关重要。

目前版本使用 Global Memory 来实现矩阵转置。而对 Global Memory 的访问是比较慢的，并且对于 Global Memory 的内存访问也是不连续的，导致 cache 命中率不高（当输入以步长 N 读取时，输入内存访问不连续；当输入连续读取时，输出写入操作步长为 N，内存访问不连续）。而每个线程块内部存在内部线程可以共享的 Share Memory，它的访问速度快于 Global Memory，并且利用 Share Memory，对 Global Memory 的读取和写入一定程度上都可以变得连续，从而一定程度提高缓存命中率。从而提升性能。

以下是使用 Share Memory 进行矩阵转置的流程示意图。先将矩阵划分成每个线程块要处理的 tile。将 tile 传入各自对应线程块的 Share Memory，找到在全局矩阵中需要写入的 tile 位置，然后按照转置后的顺序进行写入：



核心实现代码如下，为了避免多个线程同时访问共享内存中同一个 bank 导致的 bank conflict（访存冲突），我们在共享内存的 tile 数组声明中 人为添加一个多余的列（如 tile[TILE_DIM][TILE_DIM + 1]），这样每一行在内存中的起始地址被“错开”，从而打破地址对齐模式，避免冲突。并且代码允许 block 小于 tile 大小，这样就能让一个线程一次执行多个元素转置。

```cpp
#define TILE_DIM 32
#define BLOCK_ROWS 8

__global__ void transposeSharedFlexibleKernel(float* A, float* AT, int n) {
    __shared__ float tile[TILE_DIM][TILE_DIM + 1];  // 避免 bank conflict

    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;

    // 以 BLOCK_ROWS 为单位分多次加载tile的内容
    for (int i = 0; i < TILE_DIM; i += BLOCK_ROWS) {
        if (x < n && (y + i) < n) {
            tile[threadIdx.y + i][threadIdx.x] = A[IDX(y + i, x, n)];
        }
    }

    __syncthreads();

    // 交换 blockIdx 的 x 和 y，实现转置效果
    x = blockIdx.y * TILE_DIM + threadIdx.x;
    y = blockIdx.x * TILE_DIM + threadIdx.y;

    for (int i = 0; i < TILE_DIM; i += BLOCK_ROWS) {
        if (x < n && (y + i) < n) {
            AT[IDX(y + i, x, n)] = tile[threadIdx.x][threadIdx.y + i];
        }
    }
}
```

实验结果如下，在 block 为（32,8），title 为 32 的情况下运行结果：

```
Transpose time: 0.4019 ms
```

只在 block（32,8）的普通转置程序运行效果：

```
Transpose time: 0.8839 ms
```

可以看到通过共享内存优化，可以将运行性能提升一倍以上。

如果将 block 变成（32,32）时，运行结果如下：

```
Transpose time: 0.5239 ms
```

当 block 变大时，一个线程负载一个元素转置，这会导致更多的线程运行，需要更多的 warp，这应该是导致性能下降的原因。

4. 实验感想

通过这次实验，我完成了 CUDA 编程入门的 hello 输出，分析了 hello 输出的规律。我实现了 CUDA 版本的矩阵转置，分析了矩阵规模和 block 大小对性能的影响，并且通过共享内存对其进行优化。对其了解了 CUDA 的底层运行逻辑，以及学习优化 CUDA 程序性能的方法。受益匪浅。